Skill

Topic 1 : To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
vennela_07@VennelaPC:~$ mkdir history_project
vennela_07@VennelaPC:~$ cd history_project
vennela_07@VennelaPC:~/history_project$ nano command_history.c
vennela_07@VennelaPC:~/history_project$ gcc command_history.c -o command_history
vennela_07@VennelaPC:~/history_project$ ./command_history
Command History Demo
Type commands. Use UP/DOWN arrows, Enter to submit, Ctrl+D to exit.

> hello
> open file
>                                                                    >
                                                                  >
Exiting command history demo.
```

```
  GNU nano 7.2                                                command_history.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define SIZE 100
#define HISTORY 10

char history[HISTORY][SIZE];
char buffer[SIZE];

int count = 0, position = 0;

void set_raw_mode(struct termios *old) {
    struct termios raw;
    tcgetattr(STDIN_FILENO, old);
    raw = *old;
    raw.c_lflag &= ~(ICANON | ECHO);
    raw.c_iflag &= ~(IXON | ICRNL);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

int main() {
    struct termios old;
    int ch, i = 0;

    if (!isatty(STDIN_FILENO)) {
        printf("Run this program in a terminal.\n");
        return 1;
```

```
    }

    set_raw_mode(&old);

    printf("Command History Demo\n");
    printf("Type commands. Use UP/DOWN arrows, Enter to submit, Ctrl+D to exit.\n\n");
    printf("> ");
    fflush(stdout);

    while (1) {
        ch = getchar();

        if (ch == 4) {
            break;
        }

        if (ch == '\n' || ch == '\r') {
            buffer[i] = '\0';
            printf("\n");

            if (i > 0) {
                if (count < HISTORY) {
                    strcpy(history[count++], buffer);
                } else {
                    for (int j = 1; j < HISTORY; j++)
                        strcpy(history[j - 1], history[j]);
                    strcpy(history[HISTORY - 1], buffer);
                }
            }

            i = 0;
            position = count;
            buffer[0] = '\0';

            printf("> ");
            fflush(stdout);
        }
        else if (ch == 127 || ch == 8) {
            if (i > 0) {
                buffer[--i] = '\0';
                    printf("\b \b");
                    fflush(stdout);
                }
          }
          else if (ch == 27) {
                int c1 = getchar();
                int c2 = getchar();

                if (c1 == '[' && c2 == 'A' && position > 0) {
                    position--;
                    strcpy(buffer, history[position]);
                    i = strlen(buffer);
                    printf("\r> %-99s\r> %s", "", buffer);
                    fflush(stdout);
                }
                else if (c1 == '[' && c2 == 'B' && position < count) {
                    position++;
                    if (position == count) {
                        i = 0;
                        buffer[0] = '\0';
                    } else {
```

```
                    strcpy(buffer, history[position]);
                    i = strlen(buffer);
                }
                printf("\r> %-99s\r> %s", "", buffer);
                fflush(stdout);
            }
        }
        else if (ch >= 32 && ch <= 126 && i < SIZE - 1) {
            buffer[i++] = ch;
            buffer[i] = '\0';
            putchar(ch);
            fflush(stdout);
        }
    }

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &old);
    printf("\nExiting command history demo.\n");
    return 0;
}
```

Topic 2 : To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
vennela_07@VennelaPC:~$ mkdir memory_lab
vennela_07@VennelaPC:~$ cd memory_lab
vennela_07@VennelaPC:~/memory_lab$ nano dynamic_memory.c
vennela_07@VennelaPC:~/memory_lab$ gcc -Wall -Wextra -g dynamic_memory.c -o dynamic_memory
vennela_07@VennelaPC:~/memory_lab$ ./dynamic_memory
Enter integers separated by spaces:
10 20 30 40 50

Dynamic array: 10 20 30 40 50
Linked list: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Number of elements: 5
Array capacity: 8
All allocated memory released.
vennela_07@VennelaPC:~/memory_lab$ valgrind --leak-check=full --track-origins=yes ./dynamic_memory
==1068== Memcheck, a memory error detector
==1068== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1068== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1068== Command: ./dynamic_memory
==1068==
Enter integers separated by spaces:
10 20 30 40 50

Dynamic array: 10 20 30 40 50
Linked list: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Number of elements: 5
Array capacity: 8
All allocated memory released.
==1068==
==1068== HEAP SUMMARY:
==1068==     in use at exit: 0 bytes in 0 blocks
==1068==   total heap usage: 10 allocs, 10 frees, 2,296 bytes allocated
==1068==
==1068== All heap blocks were freed -- no leaks are possible
==1068==
==1068== For lists of detected and suppressed errors, rerun with: -s
==1068== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
  GNU nano 7.2                                                  dynamic_memory.c

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include <ctype.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int append_array(int **arr, size_t *size, size_t *capacity, int value) {
    if (*size == *capacity) {
        size_t new_capacity = (*capacity == 0) ? 4 : *capacity * 2;

        if (new_capacity < *capacity ||
            new_capacity > (size_t)-1 / sizeof(int)) {
            return 0;
        }

        int *temp = realloc(*arr, new_capacity * sizeof(int));
        if (temp == NULL) {
            return 0;
        }

        *arr = temp;
        *capacity = new_capacity;
    }

    (*arr)[(*size)++] = value;
    return 1;
}

int append_list(Node **head, int value) {
    Node *node = malloc(sizeof(Node));

    if (node == NULL) {
        return 0;
    }
```

Topic 3 : To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
vennela_07@VennelaPC:~$ mkdir tokenizer_project
vennela_07@VennelaPC:~$ cd tokenizer_project
vennela_07@VennelaPC:~/tokenizer_project$ nano token_parser.c
vennela_07@VennelaPC:~/tokenizer_project$ gcc -Wall -Wextra token_parser.c -o token_parser
vennela_07@VennelaPC:~/tokenizer_project$ ./token_parser
Enter an expression:
sum = value1 + 25;
Debug: Token 1 = [sum]
Debug: Token 2 = [=]
Debug: Token 3 = [value1]
Debug: Token 4 = [+]
Debug: Token 5 = [25]
Debug: Token 6 = [;]

--- Token Stream ---
Token 1: Value = sum            Type = WORD/NUMBER
Token 2: Value = =              Type = DELIMITER
Token 3: Value = value1         Type = WORD/NUMBER
Token 4: Value = +              Type = DELIMITER
Token 5: Value = 25             Type = WORD/NUMBER
Token 6: Value = ;              Type = DELIMITER

Validation successful.
Total tokens: 6
```

```
vennela_07@VennelaPC:~/tokenizer_project$ ./token_parser
Enter an expression:
a @ b
Debug: Token 1 = [a]
Debug: Token 2 = [@]
Debug: Token 3 = [b]

--- Token Stream ---
Token 1: Value = a                Type = WORD/NUMBER
Token 2: Value = @                Type = UNKNOWN
Validation failed: Unknown character.
```

Topic 4 : To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
vennela_07@VennelaPC:~$ mkdir parser_project
vennela_07@VennelaPC:~$ mkdir parser_project
mkdir: cannot create directory ‘parser_project’: File exists
vennela_07@VennelaPC:~$ cd parser_project
vennela_07@VennelaPC:~/parser_project$ nano command_parser.c
vennela_07@VennelaPC:~/parser_project$ gcc -Wall -Wextra command_parser.c -o command_parser
vennela_07@VennelaPC:~/parser_project$ ./command_parser
Enter a command (or 'quit' to exit):
> ./command_parser

Parse tree:
|-- COMMAND
  |-- ./command_parser

Execution structure:
Command: ./command_parser
Arguments:
Syntax validation successful.

> echo hello world

Parse tree:
|-- COMMAND
  |-- echo
    |-- hello
      |-- world

Execution structure:
Command: echo
Arguments: hello world
Syntax validation successful.
```

```
> Empty command: nothing to parse.

Parse tree:
|-- COMMAND
  |-- Empty
    |-- command:
      |-- nothing
        |-- to
          |-- parse.

Execution structure:
Command: Empty
Arguments: command: nothing to parse.
Syntax validation successful.

> list files
show
Parse tree:
|-- COMMAND
  |-- list
    |-- files

Execution structure:
Command: list
Arguments: files
Syntax validation successful.

> 123
quit
Parse tree:
|-- COMMAND
  |-- show
    |-- 123

Execution structure:
Command: show
Arguments: 123
Syntax validation successful.

> quit

Parse tree:
|-- COMMAND
  |-- quitquit
|-- COMMAND
  |-- quitquit

Execution structure:
Command: quitquit
Arguments:
Syntax validation successful.
```